Contents

System Design Interview Questions **5**
Complete Solutions for Top Tech Company Interviews . . . 5

Table of Contents **5**

1. Design a URL Shortener (TinyURL) **5**
Requirements . . . 5
Functional . . . 5
Non-Functional . . . 5
Capacity Estimation . . . 5
System Design . . . 6
High-Level Architecture . . . 6
URL Encoding . . . 6
Database Schema . . . 7
API Design . . . 7
Redirection Flow . . . 8
Key Design Decisions . . . 8
Scaling Strategies . . . 8

2. Design Twitter **8**
Requirements . . . 8
Functional . . . 8
Non-Functional . . . 8
Capacity Estimation . . . 9
System Design . . . 9
High-Level Architecture . . . 9
Timeline Generation . . . 10
Database Schema . . . 10
Tweet Posting Flow . . . 11
Timeline Read Flow . . . 11
Handling Celebrity Problem . . . 11

3. Design Instagram **12**
Requirements . . . 12
Functional . . . 12
Non-Functional . . . 12
Capacity Estimation . . . 12
System Design . . . 12
High-Level Architecture . . . 12
Photo Upload Flow . . . 13
Storage Architecture . . . 13
News Feed Generation . . . 13
Feed Ranking . . . 14

4. Design Facebook News Feed **14**
Requirements . . . 14
Functional . . . 14
Non-Functional . . . 14
Feed Generation Architecture . . . 14
Push Model for Close Friends . . . 14
Pull Model for Others . . . 14
Ranking System . . . 15
Feed Ranking Factors . . . 15

5. Design WhatsApp/Messenger **15**

Requirements . . . 15
Functional . . . 15
Non-Functional . . . 15
Capacity Estimation . . . 16
System Design . . . 16
High-Level Architecture . . . 16
WebSocket Connection Management . . . 16
Message Flow . . . 17
Message Storage . . . 17
Group Messaging . . . 17
End-to-End Encryption . . . 17
Presence Service . . . 18

6. Design YouTube **18**
Requirements . . . 18
Functional . . . 18
Non-Functional . . . 18
Capacity Estimation . . . 18
System Design . . . 19
High-Level Architecture . . . 19
Video Upload Pipeline . . . 19
Video Streaming . . . 20
CDN Distribution . . . 20
Recommendation System . . . 20

7. Design Netflix **21**
Requirements . . . 21
Functional . . . 21
Non-Functional . . . 21
System Design . . . 21
High-Level Architecture . . . 21
Open Connect Appliances (OCA) . . . 21
Playback Flow . . . 22
Recommendation System . . . 22

8. Design Uber/Lyft **22**
Requirements . . . 22
Functional . . . 22
Non-Functional . . . 22
Capacity Estimation . . . 23
System Design . . . 23
High-Level Architecture . . . 23
Location Service . . . 23
Matching Algorithm . . . 24
Trip Lifecycle . . . 24
Surge Pricing . . . 24

9. Design a Web Crawler **25**
Requirements . . . 25
Functional . . . 25
Non-Functional . . . 25
System Design . . . 25
High-Level Architecture . . . 25
URL Frontier . . . 25
Politeness . . . 26
Duplicate Detection . . . 26
Content Storage . . . 26

10. Design Dropbox/Google Drive **27**
Requirements . . . 27
Functional . . . 27
Non-Functional . . . 27
System Design . . . 27
High-Level Architecture . . . 27
File Chunking . . . 27
Sync Algorithm . . . 27
Conflict Resolution . . . 28
Metadata Database . . . 28

11. Design a Rate Limiter **29**
Requirements . . . 29
Functional . . . 29
Non-Functional . . . 29
Algorithms . . . 29
Token Bucket . . . 29
Sliding Window Log . . . 29
Sliding Window Counter . . . 30
System Design . . . 30
Response Headers . . . 30

12. Design a Key-Value Store **30**
Requirements . . . 30
Functional . . . 30
Non-Functional . . . 31
System Design (Dynamo-style) . . . 31
Architecture . . . 31
Data Partitioning (Consistent Hashing) . . . 31
Replication . . . 31
Write Path . . . 31
Read Path . . . 31
Conflict Resolution . . . 31

13. Design a Distributed Cache **32**
Requirements . . . 32
Functional . . . 32
Non-Functional . . . 32
System Design . . . 32
Architecture (Redis Cluster style) . . . 32
Data Distribution . . . 32
Cache Eviction . . . 33
High Availability . . . 33

14. Design a Search Autocomplete System **33**
Requirements . . . 33
Functional . . . 33
Non-Functional . . . 33
System Design . . . 33
Architecture . . . 33
Trie Data Structure . . . 34
Building the Trie . . . 34
Query Flow . . . 34
Updating Suggestions . . . 34

15. Design Yelp/Nearby Places **35**
Requirements . . . 35

Functional . . . 35
Non-Functional . . . 35
System Design . . . 35
Architecture . . . 35
Geospatial Indexing . . . 35
Search Query . . . 36

16. Design a Notification System **36**
Requirements . . . 36
Functional . . . 36
Non-Functional . . . 36
System Design . . . 36
Notification Flow . . . 37
User Preferences . . . 37

17. Design an Online Ticketing System **37**
Requirements . . . 37
Functional . . . 37
Non-Functional . . . 37
System Design . . . 38
Handling Concurrency . . . 38
Booking Flow . . . 38
Queue System for High Demand . . . 38

18. Design Pastebin **38**
Requirements . . . 38
Functional . . . 38
Non-Functional . . . 39
System Design . . . 39
Key Generation . . . 39

19. Design a Unique ID Generator **39**
Requirements . . . 39
Functional . . . 39
Non-Functional . . . 39
Approaches . . . 39
1. UUID . . . 39
2. Database Auto-Increment . . . 40
3. Twitter Snowflake . . . 40
Implementation (Snowflake) . . . 40

20. Design a Payment System **41**
Requirements . . . 41
Functional . . . 41
Non-Functional . . . 41
System Design . . . 41
Payment Flow . . . 41
Idempotency . . . 42
Double-Entry Ledger . . . 42
Reconciliation . . . 42

Quick Reference: System Design Checklist **42**
For Every Design . . . 42

System Design Interview Questions

Complete Solutions for Top Tech Company Interviews

Table of Contents

1. Design a URL Shortener (TinyURL)
2. Design Twitter
3. Design Instagram
4. Design Facebook News Feed
5. Design WhatsApp/Messenger
6. Design YouTube
7. Design Netflix
8. Design Uber/Lyft
9. Design a Web Crawler
10. Design Dropbox/Google Drive
11. Design a Rate Limiter
12. Design a Key-Value Store
13. Design a Distributed Cache
14. Design a Search Autocomplete System
15. Design Yelp/Nearby Places
16. Design a Notification System
17. Design an Online Ticketing System
18. Design Pastebin
19. Design a Unique ID Generator
20. Design a Payment System

---

1. Design a URL Shortener (TinyURL)

Requirements

Functional

- Given a URL, generate a shorter unique alias
- Redirect short URL to original URL
- Custom short URLs (optional)
- Expiration time (optional)
- Analytics (click count, location)

Non-Functional

- High availability
- Low latency redirection
- Short URLs should not be predictable

Capacity Estimation

```
Assumptions:
- 100 million URLs created per month
- Read:Write ratio = 100:1
- URL stored for 5 years

Traffic:
```

```
- Write: 100M / (30 * 24 * 3600) = ~40 URLs/second
- Read: 40 * 100 = 4,000 redirects/second

Storage:
- Each URL: 500 bytes
- 5 years: 100M * 12 * 5 = 6 billion URLs
- Total: 6B * 500 bytes = 3 TB

Bandwidth:
- Write: 40 * 500 bytes = 20 KB/s
- Read: 4,000 * 500 bytes = 2 MB/s

Cache:
- 20% of URLs generate 80% of traffic
- Cache 20% of daily reads
- Daily reads: 4,000 * 86,400 = 350 million
- Cache size: 350M * 0.2 * 500 bytes = 35 GB
```

System Design

High-Level Architecture

```
                                  +---------------+
                                  |    CDN        |
                                  +---------------+
                                         |
+--------+    +-------------+    +---------+---------+
| Client | -> | Load        | -> | Application      |
+--------+    | Balancer    |    | Servers          |
              +-------------+    +--------+---------+
                                          |
                    +---------------------+---------------------+
                    |                     |                     |
             +------v------+       +------v------+       +------v------+
             |    Cache    |       |  Key Gen    |       |  Analytics  |
             |   (Redis)   |       |  Service    |       |  Service    |
             +-------------+       +-------------+       +-------------+
                    |                     |
             +------v---------------------v------+
             |          Database (SQL)           |
             |  - URL mappings                   |
             |  - User data                      |
             +-----------------------------------+
```

URL Encoding

Option 1: Base62 Encoding

```
Characters: [a-z, A-Z, 0-9] = 62 characters
6 characters: 62^6 = ~57 billion combinations
7 characters: 62^7 = ~3.5 trillion combinations

Example: abc123 -> base62 decode -> integer -> original URL
```

Option 2: MD5/SHA256 Hash

```
1. Hash the original URL
2. Take first 6-7 characters
3. Handle collisions by appending sequence number
```

Option 3: Pre-generated Keys

```
1. Generate keys in advance
2. Store in separate Key Generation Service
3. Mark keys as used/unused
4. No collision handling needed
```

Database Schema

```sql
-- URL table
CREATE TABLE urls (
    id BIGINT PRIMARY KEY,
    short_url VARCHAR(10) UNIQUE NOT NULL,
    original_url TEXT NOT NULL,
    user_id BIGINT,
    created_at TIMESTAMP DEFAULT NOW(),
    expires_at TIMESTAMP,
    click_count BIGINT DEFAULT 0
);

-- Key table (for pre-generated keys)
CREATE TABLE keys (
    key_value VARCHAR(10) PRIMARY KEY,
    used BOOLEAN DEFAULT FALSE
);

-- Analytics table
CREATE TABLE analytics (
    id BIGINT PRIMARY KEY,
    short_url VARCHAR(10),
    clicked_at TIMESTAMP,
    ip_address VARCHAR(45),
    user_agent TEXT,
    referrer TEXT
);

CREATE INDEX idx_short_url ON urls(short_url);
CREATE INDEX idx_analytics_short_url ON analytics(short_url);
```

API Design

```
POST /api/v1/shorten
Request:
{
    "original_url": "https://example.com/very/long/url",
    "custom_alias": "mylink",  // optional
    "expires_in": 86400        // optional, seconds
}
Response:
{
    "short_url": "https://tinyurl.com/abc123",
    "original_url": "https://example.com/very/long/url",
    "expires_at": "2024-01-16T10:00:00Z"
}

GET /{short_url}
Response: 301/302 Redirect to original URL
```

```
GET /api/v1/stats/{short_url}
Response:
{
    "short_url": "abc123",
    "click_count": 1542,
    "created_at": "2024-01-15T10:00:00Z"
}
```

Redirection Flow

```
1. Client requests short URL
2. Load balancer routes to app server
3. Check cache for mapping
4. If cache miss, query database
5. Update cache
6. Return 301 (permanent) or 302 (temporary) redirect
7. Async: Log analytics
```

Key Design Decisions

301 vs 302 Redirect: - 301 Permanent: Browser caches, less server load, less accurate analytics - 302 Temporary: No browser caching, accurate analytics

Database Choice: - SQL: Strong consistency, good for URL mappings - NoSQL (DynamoDB): Better scaling, eventual consistency acceptable

Scaling Strategies

1. **Database sharding:** By hash of short URL
2. **Read replicas:** For high read traffic
3. **Caching:** Cache popular URLs in Redis
4. **CDN:** Cache redirects at edge
5. **Key Generation Service:** Separate service to avoid bottleneck

---

2. Design Twitter

Requirements

Functional

- Post tweets (280 characters)
- Follow/Unfollow users
- Timeline: Home (followed users) and User (own tweets)
- Search tweets
- Like, Retweet, Reply
- Notifications

Non-Functional

- High availability
- Low latency timeline generation (<200ms)
- Eventual consistency acceptable
- Handle viral tweets (celebrity problem)

Capacity Estimation

```
Users:
- 300 million monthly active users
- 150 million daily active users

Tweets:
- 500 million tweets/day
- Average user: 3 tweets/day
- 5% users are heavy posters

Traffic:
- Write: 500M / 86,400 = ~6,000 tweets/second
- Read: Each user reads 100 tweets/day
- Read: 150M * 100 / 86,400 = ~175,000 reads/second

Storage (per tweet):
- Tweet ID: 8 bytes
- User ID: 8 bytes
- Text: 280 bytes
- Timestamp: 8 bytes
- Likes/Retweets: 8 bytes
- Total: ~500 bytes per tweet

Daily storage: 500M * 500 bytes = 250 GB/day
Yearly: ~90 TB
```

System Design

High-Level Architecture

```
+--------+     +-------------+     +------------------+
| Client | --> | Load        | --> | API Gateway      |
+--------+     | Balancer    |     +------------------+
               +-------------+            |
                                          v
    +------------+------------+------------+------------+
    |            |            |            |            |
+---v---+   +----v----+  +----v----+  +----v----+  +----v----+
| Tweet |   | Timeline|  | User    |  | Search  |  | Notif   |
|Service|   | Service |  | Service |  | Service |  | Service |
+---+---+   +----+----+  +----+----+  +----+----+  +----+----+
    |            |            |            |            |
    +------------v------------+            |            |
                 |                         |            |
    +------------v------------+     +------v------+     |
    |     Cache (Redis)       |     | Elasticsearch|    |
    | - Timeline cache        |     +-------------+     |
    | - User cache            |                         |
    +------------+------------+                         |
                 |                                      |
    +------------v------------+     +-------------------v+
    |   Message Queue (Kafka) |     |   Notification    |
    +------------+------------+     |   Queue           |
                 |                  +-------------------+
    +------------v------------+
    |      Database           |
```

```
    | - Tweets (Cassandra)    |
    | - Users (MySQL)         |
    | - Followers (MySQL)     |
    +-------------------------+
```

Timeline Generation

Approach 1: Pull Model (Fan-out on Read)

```
When user opens timeline:
1. Get list of followed users
2. For each followed user, get recent tweets
3. Merge and sort by time
4. Return top N tweets

Pros:
- Simple
- Works well for users with few followers

Cons:
- Slow for users following many accounts
- High read latency
```

Approach 2: Push Model (Fan-out on Write)

```
When user posts tweet:
1. Get list of followers
2. Push tweet to each follower's timeline cache
3. When user opens timeline, read from cache

Pros:
- Fast reads
- Pre-computed timelines

Cons:
- Celebrity problem (millions of followers)
- Storage intensive
- Delayed for new follows
```

Approach 3: Hybrid Model (Recommended)

```
For regular users:
- Use push model
- Fan-out tweets to follower timelines

For celebrities (>1M followers):
- Use pull model
- Fetch celebrity tweets on timeline read
- Merge with pre-computed timeline
```

Database Schema

```sql
-- Users table (MySQL)
CREATE TABLE users (
    user_id BIGINT PRIMARY KEY,
    username VARCHAR(50) UNIQUE,
    email VARCHAR(255),
    created_at TIMESTAMP,
    followers_count INT,
```

```sql
    following_count INT
);

-- Followers table (MySQL)
CREATE TABLE followers (
    follower_id BIGINT,
    followee_id BIGINT,
    created_at TIMESTAMP,
    PRIMARY KEY (follower_id, followee_id)
);
CREATE INDEX idx_followee ON followers(followee_id);

-- Tweets table (Cassandra)
-- Partition by user_id for user timeline
CREATE TABLE tweets (
    tweet_id BIGINT,
    user_id BIGINT,
    content TEXT,
    created_at TIMESTAMP,
    likes_count INT,
    retweets_count INT,
    PRIMARY KEY (user_id, tweet_id)
) WITH CLUSTERING ORDER BY (tweet_id DESC);

-- Home Timeline (Redis)
Key: timeline:{user_id}
Value: Sorted Set of tweet_ids with timestamp as score
```

Tweet Posting Flow

```
1. Client sends tweet to API Gateway
2. Tweet Service validates and stores tweet
3. Tweet sent to Kafka
4. Fan-out Service consumes from Kafka
5. For each follower (non-celebrity):
   - Add tweet to follower's timeline cache
6. Update search index (Elasticsearch)
7. Send notifications if mentioned
```

Timeline Read Flow

```
1. Client requests timeline
2. Read from Redis timeline cache
3. If user follows celebrities:
   - Fetch celebrity tweets from DB
   - Merge with cached timeline
4. Return sorted tweets
```

Handling Celebrity Problem

```
Celebrity detection:
- Followers > 1 million = celebrity
- Store in separate list

Timeline generation for celebrity followers:
1. Get pre-computed timeline from cache
2. Get list of followed celebrities
```

```
3. Fetch recent tweets from each celebrity (parallel)
4. Merge-sort all tweets by timestamp
5. Cache merged result briefly (30 seconds)
```

---

3. Design Instagram

Requirements

Functional

- Upload photos/videos
- Follow users
- News feed
- Like and comment
- Stories (24-hour content)
- Direct messaging
- Search (users, hashtags, locations)

Non-Functional

- High availability
- Low latency feed
- Reliability (no photo loss)
- Handle high read:write ratio (100:1)

Capacity Estimation

```
Users:
- 500 million daily active users
- Average user: 5 photos/videos viewed per session
- 5 sessions per day

Uploads:
- 100 million photos/videos per day
- Average photo: 2 MB
- Average video: 50 MB

Storage:
- Photos: 100M * 0.8 * 2 MB = 160 TB/day
- Videos: 100M * 0.2 * 50 MB = 1 PB/day
- Total: ~1.2 PB/day

Traffic:
- Uploads: 100M / 86,400 = ~1,200/second
- Views: 500M * 5 * 5 / 86,400 = ~145,000/second
```

System Design

High-Level Architecture

```
+--------+     +-------+     +-------------+
| Client | --> |  CDN  | --> | Load        |
+--------+     +-------+     | Balancer    |
                             +------+------+
                                    |
         +--------------------------+--------------------------+
```

```
     |              |           |           |              |
+----v----+   +-----v-----+ +---v---+ +-----v-----+  +-----v-----+
|  Upload |   |   Feed    | | User  | |  Search   |  |  Stories  |
| Service |   |  Service  | |Service| |  Service  |  |  Service  |
+---------+   +-----------+ +-------+ +-----------+  +-----------+
     |              |           |           |              |
+----v----+         |      +----v----+  +---v----+    +----v----+
| Object  |         |      |  User   |  | Elastic|    |  Redis  |
| Storage |         |      |   DB    |  | Search |    |  Cache  |
| (S3)    |         |      | (MySQL) |  +--------+    +---------+
+---------+         |      +---------+
                    |
          +---------v----------+
          | Feed Generation    |
          | - Cassandra        |
          | - Redis Timeline   |
          +--------------------+
```

Photo Upload Flow

```
1. Client uploads photo to Upload Service
2. Generate unique photo ID
3. Store photo metadata in MySQL
4. Upload image to S3
5. Trigger async processing:
   a. Generate thumbnails (different sizes)
   b. Apply filters if requested
   c. Extract EXIF data
   d. Detect faces for tagging
   e. Index for search
6. Return success with photo URL
```

Storage Architecture

```
Photo Storage Tiers:
- Hot: Recent photos, high-performance SSD
- Warm: 1-6 months old, standard storage
- Cold: >6 months, archive storage (Glacier)

Multiple resolutions:
- Original: Full quality backup
- Large: 1080p for desktop
- Medium: 640p for mobile
- Thumbnail: 150p for previews

CDN Distribution:
- Popular content at edge
- Geographic distribution
- Reduce origin load
```

News Feed Generation

Approach: Hybrid Push-Pull

```
On post:
1. Store post in database
2. Fan-out to active followers' feeds (push)
3. For users following many accounts, use pull
```

```
On feed read:
1. Get pre-computed feed from cache
2. For celebrities followed, fetch recent posts
3. Merge and rank
4. Apply machine learning ranking
5. Return personalized feed
```

Feed Ranking

```
Ranking factors:
- Recency
- User engagement (likes, comments)
- Relationship strength (interaction history)
- Content type preference
- Post popularity
- Time spent viewing similar content

ML model:
- Feature extraction
- Engagement prediction
- Ranking score computation
```

---

4. Design Facebook News Feed

Requirements

Functional

- View feed of posts from friends and followed pages
- Post text, images, videos
- Like, comment, share
- Personalized ranking
- Real-time updates

Non-Functional

- Low latency (<500ms)
- High availability
- Handle billions of daily requests
- Support viral content

Feed Generation Architecture

Push Model for Close Friends

```
When user posts:
1. Identify close friends (frequent interaction)
2. Push to their pre-computed feeds
3. Store in Redis sorted sets
```

Pull Model for Others

```
On feed request:
1. Get pre-computed feed from cache
2. Fetch recent posts from wider network
3. Apply ranking algorithm
```

```
4. Merge and return top posts
```

Ranking System

```
                    +------------------+
                    |  Feature Store   |
                    +--------+---------+
                             |
+--------+     +-------------v--------------+     +--------+
|  Posts | --> |    Ranking Model           | --> |  Feed  |
+--------+     | - Engagement prediction    |     +--------+
               | - Relevance scoring        |
               | - Diversity injection      |
               +----------------------------+
```

Feed Ranking Factors

```
1. Affinity Score
   - Interaction frequency
   - Relationship type
   - Recency of interaction

2. Edge Weight
   - Type of interaction (comment > like)
   - Duration of engagement

3. Time Decay
   - Newer posts ranked higher
   - Exponential decay function

4. Content Type
   - User preferences
   - Platform priorities

5. Engagement Velocity
   - How fast post is gaining engagement
   - Viral content detection
```

---

5. Design WhatsApp/Messenger

Requirements

Functional

- One-on-one messaging
- Group messaging (up to 256 members)
- Read receipts (sent, delivered, read)
- Online/last seen status
- Media sharing (images, videos, documents)
- End-to-end encryption
- Push notifications

Non-Functional

- Real-time delivery (<100ms)
- High availability (99.99%)

- Message ordering guaranteed
- Handle 100 billion messages/day

Capacity Estimation

```
Users: 2 billion monthly, 500 million daily
Messages: 100 billion/day

Traffic:
- Messages: 100B / 86,400 = ~1.2 million/second
- Connections: 500M concurrent WebSocket connections

Storage:
- Average message: 100 bytes
- Daily: 100B * 100 bytes = 10 TB
- 30-day retention: 300 TB
```

System Design

High-Level Architecture

```
+--------+     +---------------+     +------------------+
| Client | <-> | Load Balancer | <-> | WebSocket        |
| (App)  |     |               |     | Gateway Servers  |
+--------+     +---------------+     +---------+---------+
                                               |
         +-----------------------------------+--------------------+
         |                    |              |                    |
    +----v----+         +-----v-----+   +-----v-----+        +-----v-----+
    | Message |         |  Presence |   |   Group   |        |   Media   |
    | Service |         |  Service  |   |  Service  |        |  Service  |
    +----+----+         +-----+-----+   +-----+-----+        +-----+-----+
         |                    |              |                    |
    +----v----+         +-----v-----+   +-----v-----+        +-----v-----+
    | Message |         |   Redis   |   |   Group   |        |   Object  |
    | Queue   |         |  (Status) |   |    DB     |        |  Storage  |
    | (Kafka) |         +-----------+   +-----------+        +-----------+
    +----+----+
         |
    +----v----+
    | Message |
    | Storage |
    | (HBase) |
    +---------+
```

WebSocket Connection Management

```
Connection Flow:
1. Client establishes WebSocket connection
2. Authenticate with session token
3. Connection mapped to user in Connection Registry
4. Heartbeat every 30 seconds to maintain connection

Connection Registry (Redis):
Key: user:{user_id}:connection
Value: {server_id, connection_id, last_active}
```

```
Server Distribution:
- Consistent hashing for user-to-server mapping
- Sticky sessions for connection affinity
```

Message Flow

```
Sending Message:
1. Client sends message via WebSocket
2. Gateway validates and forwards to Message Service
3. Message stored in database
4. Message sent to Kafka topic
5. Consumer reads message
6. Check recipient's connection status
7. If online: Push via WebSocket
   If offline: Store for later + push notification
8. Return delivery acknowledgment

Message States:
- Sent: Server received
- Delivered: Recipient's device received
- Read: Recipient opened message
```

Message Storage

```
HBase Schema:
Row Key: {conversation_id}_{timestamp}_{message_id}

Column Family: message
  - sender_id
  - content (encrypted)
  - media_url
  - status

Column Family: metadata
  - sent_at
  - delivered_at
  - read_at
```

Group Messaging

```
Group Structure:
- Group ID
- Admin list
- Member list (max 256)
- Settings (muted, notifications)

Message Delivery:
1. Sender sends to group
2. Fetch all group members
3. For each member:
   - Check online status
   - Deliver via WebSocket or queue for later
4. Track delivery status per member
```

End-to-End Encryption

```
Signal Protocol:
1. Each user generates public/private key pair
```

```
2. Public keys stored on server
3. Messages encrypted with recipient's public key
4. Only recipient can decrypt with private key
5. Server cannot read message content

Key Exchange:
- Diffie-Hellman key exchange
- Perfect forward secrecy
- New session keys periodically
```

Presence Service

```
Online Status:
- Heartbeat every 30 seconds
- If no heartbeat for 60 seconds, mark offline
- Store in Redis with TTL

Last Seen:
- Update on each message/action
- Respect privacy settings

Typing Indicator:
- Client sends typing start/stop events
- Broadcast to conversation participants
- Auto-expire after 5 seconds
```

---

6. Design YouTube

Requirements

Functional

- Upload videos
- Watch videos (streaming)
- Search videos
- Like, comment, subscribe
- Recommendations
- Live streaming

Non-Functional

- High availability
- Low latency video playback
- Handle viral videos
- Support multiple resolutions

Capacity Estimation

```
Users: 2 billion monthly, 1 billion daily
Videos watched: 1 billion/day
Videos uploaded: 500,000/day

Video Storage:
- Average video: 5 minutes, 100 MB (compressed)
- Daily uploads: 500K * 100 MB = 50 TB
- Multiple resolutions (4): 50 TB * 4 = 200 TB/day
```

```
Bandwidth:
- Watch: 1B * 100 MB = 100 PB/day
- Peak: 100 PB / 86,400 = ~1.2 TB/second
```

System Design

High-Level Architecture

```
                        +------------------+
                        |       CDN        |
                        | (Video Delivery) |
                        +--------+---------+
                                 |
+--------+     +-------+     +---v---+     +------------------+
| Client | --> |  LB   | --> |  API  | --> | Video Processing |
+--------+     +-------+     |Gateway|     |    Pipeline      |
                             +---+---+     +--------+---------+
                                 |                  |
         +-----------------------+                  |
         |          |           |                  |
    +----v---+  +----v---+  +----v---+      +-------v-------+
    | Video  |  | Search |  | User   |      | Object        |
    |Metadata|  | Service|  | Service|      | Storage (S3)  |
    | Service|  +--------+  +--------+      +---------------+
    +---+----+       |
        |            |
    +---v----+  +----v--------+
    |Cassandra|  |Elasticsearch|
    +--------+  +-------------+
```

Video Upload Pipeline

```
1. Upload Request
   - Client requests upload URL
   - Server returns pre-signed S3 URL

2. Direct Upload
   - Client uploads directly to S3
   - Bypasses application servers

3. Processing Trigger
   - S3 event triggers processing pipeline
   - Message sent to Kafka

4. Video Processing (Async)
   a. Transcoding to multiple resolutions
      - 240p, 360p, 480p, 720p, 1080p, 4K
   b. Generate thumbnails
   c. Extract audio
   d. Create adaptive bitrate files (HLS/DASH)
   e. Content moderation (ML)
   f. Extract metadata

5. Completion
   - Update video status to "published"
   - Index for search
```

```
   - Notify user
```

Video Streaming

```
Adaptive Bitrate Streaming:
1. Video divided into small segments (2-10 seconds)
2. Each segment available in multiple qualities
3. Client monitors bandwidth
4. Requests appropriate quality segments
5. Seamless quality switching

Protocols:
- HLS (HTTP Live Streaming): Apple
- DASH (Dynamic Adaptive Streaming): Cross-platform

Manifest File (m3u8):
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=800000,RESOLUTION=640x360
360p.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1400000,RESOLUTION=1280x720
720p.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=2800000,RESOLUTION=1920x1080
1080p.m3u8
```

CDN Distribution

```
CDN Strategy:
1. Popular videos cached at edge globally
2. Less popular: Regional cache
3. Rare videos: Origin fetch

Cache Warming:
- Predict viral videos
- Pre-push to edge locations

Geographic Distribution:
- Edge servers in major cities
- ISP partnerships for peering
```

Recommendation System

```
+-------------+     +------------------+     +-------------+
| User        | --> | Candidate        | --> | Ranking     |
| Signals     |     | Generation       |     | Model       |
+-------------+     +------------------+     +-------------+
                           |                       |
                    +------v------+         +------v------+
                    | Collaborative|         | Deep        |
                    | Filtering   |         | Learning    |
                    +-------------+         +-------------+

Signals:
- Watch history
- Search history
- Likes/Dislikes
- Watch time
- Demographics
```

```
Features:
- Video metadata
- Channel info
- Engagement metrics
- Freshness
```

---

7. Design Netflix

Requirements

Functional

- Browse content catalog
- Search
- Stream video
- Personalized recommendations
- Multiple profiles per account
- Download for offline
- Resume playback

Non-Functional

- 99.99% availability
- Low latency playback start (<2 seconds)
- Smooth streaming (no buffering)
- Global distribution

System Design

High-Level Architecture

```
                          +--------+
                          |  CDN   |
                          | (OCA)  |
                          +----+---+
                               |
+--------+     +-------+     +-+--+     +------------------+
| Client | --> |  LB   | --> |API | --> | Microservices    |
| (App)  |     |       |     |GW  |     | (AWS/Cloud)      |
+--------+     +-------+     +----+     +------------------+
                                                |
         +-----------------------------------+
         |            |           |           |
    +----v----+  +----v----+  +---v---+  +----v----+
    | Content |  | User    |  |Playback|  |Recommend|
    | Service |  | Service |  |Service |  |  Engine |
    +---------+  +---------+  +--------+  +---------+
```

Open Connect Appliances (OCA)

Netflix's custom CDN:

```
Strategy:
1. Content cached at ISP locations
2. 95% of traffic served from OCA
3. Reduces ISP bandwidth costs
```

```
4. Low latency for users

Content Placement:
- Popularity-based caching
- Regional preferences
- Predict upcoming demand
```

Playback Flow

```
1. User selects content
2. Client requests playback URL
3. Playback Service determines:
   - User's location
   - Available OCAs
   - Network conditions
4. Returns manifest with OCA URLs
5. Client downloads manifest
6. Client fetches segments from nearest OCA
7. Adaptive bitrate based on bandwidth
```

Recommendation System

```
Netflix's approach:
1. Row-based layout (categories)
2. Each row is a ranked list
3. Rows themselves are ranked

Algorithms:
- Collaborative filtering
- Content-based filtering
- Deep learning models
- A/B testing

Personalization:
- Different homepage per user
- Artwork personalization
- Trending adjustments
```

---

8. Design Uber/Lyft

Requirements

Functional

- Rider: Request ride, track driver, pay
- Driver: Accept rides, navigate, receive payment
- Match riders with nearby drivers
- Real-time location tracking
- Fare estimation and calculation
- Ratings and reviews

Non-Functional

- Low latency matching (<1 second)
- High availability
- Real-time location updates

- Handle traffic spikes (events, weather)

Capacity Estimation

```
Daily rides: 20 million
Active drivers: 5 million
Active riders: 100 million

Location updates:
- Each driver: Every 3 seconds
- 5M drivers * 20 updates/minute = 100M updates/minute
- ~1.7 million updates/second

Matching requests:
- 20M rides / 86,400 = ~230 requests/second
- Peak: 5x = ~1,200 requests/second
```

System Design

High-Level Architecture

```
+----------+     +-------+     +------------------+
| Rider    | --> |  LB   | --> | API Gateway      |
| App      |     +-------+     +--------+---------+
+----------+                            |
                    +-------------------+-----------------+
+----------+        |           |       |         |       |
| Driver   | --+    |           |       |         |       |
| App      |   |  +-v------+ +--v---+ +-v-----+ +-v-----+ +-v-------+
+----------+   |  |Location| | Trip | |Payment| |Pricing| |Matching |
               |  |Service | |Service| |Service| |Service| |Service  |
               |  +---+----+ +--+---+ +---+---+ +---+---+ +----+----+
               |      |         |         |         |          |
               |  +---v---------v---------v---------v----------v---+
               |  |                  Kafka                         |
               |  +---+-------------------------------------------+
               |      |
               +------+
                      |
          +-----------+-----------+
          |           |           |
     +----v----+ +----v----+ +----v----+
     | Redis   | | PostGIS | | Payment |
     | (Geo)   | | (Trips) | | Gateway |
     +---------+ +---------+ +---------+
```

Location Service

```
Driver Location Storage:
- Use geospatial index (Redis with GeoHash)
- Update every 3 seconds
- Store: {driver_id, lat, lng, status, heading, speed}

GeoHash:
- Encode lat/lng into string
- Nearby locations have similar prefixes
- Efficient range queries
```

```
Redis Commands:
GEOADD drivers {lng} {lat} driver_123
GEORADIUS drivers {lng} {lat} 5 km
```

Matching Algorithm

```
1. Rider requests ride
2. Get pickup location
3. Query nearby available drivers (radius)
4. Rank drivers by:
   - Distance/ETA
   - Driver rating
   - Vehicle type match
   - Historical acceptance rate
5. Send request to top driver
6. If declined/timeout, send to next
7. Confirm match

Optimization:
- Pre-compute supply in areas
- Batch matching during high demand
- Surge pricing to balance supply/demand
```

Trip Lifecycle

```
States:
1. REQUESTED: Rider requested ride
2. DRIVER_ASSIGNED: Driver accepted
3. EN_ROUTE_PICKUP: Driver heading to pickup
4. ARRIVED_PICKUP: Driver at pickup location
5. TRIP_STARTED: Rider picked up
6. TRIP_COMPLETED: Arrived at destination
7. PAYMENT_PROCESSED: Payment successful

Each state change:
- Update database
- Notify rider/driver
- Update location tracking
```

Surge Pricing

```
Algorithm:
1. Define geographic zones
2. Calculate supply (available drivers)
3. Calculate demand (recent requests)
4. Surge multiplier = f(demand/supply)
5. Update prices in real-time

Example:
- Normal: $10 base
- Surge 1.5x: $15 base
- Surge 2.0x: $20 base
```

---

9. Design a Web Crawler

Requirements

Functional

- Crawl web pages starting from seed URLs
- Extract links and content
- Store pages for indexing
- Handle robots.txt
- Detect and handle duplicates

Non-Functional

- Politeness (rate limiting per domain)
- Scalability (billions of pages)
- Freshness (re-crawl updated pages)
- Robustness (handle failures)

System Design

High-Level Architecture

```
+-------------+     +-------------+     +----------------+
| URL         | --> | URL         | --> | Fetcher        |
| Seed        |     | Frontier    |     | Workers        |
+-------------+     +------+------+     +-------+--------+
                           |                    |
                    +------v------+      +------v--------+
                    | URL         |      | Content       |
                    | Deduplication|      | Parser        |
                    +-------------+      +-------+-------+
                                                 |
                    +---------------------------+
                    |                           |
             +------v------+            +-------v-------+
             | Link        |            | Content       |
             | Extractor   |            | Storage       |
             +------+------+            +---------------+
                    |
             +------v------+
             | URL         |
             | Filter      |
             +------+------+
                    |
             +------v------+
             | Add to      |
             | Frontier    |
             +-------------+
```

URL Frontier

```
Purpose:
- Queue of URLs to crawl
- Prioritization
- Politeness (per-domain rate limiting)

Implementation:
Front Queues: Priority-based
```

```
Back Queues: Per-host queues

+------------------+
| Priority Queues  |
| - High priority  |
| - Medium priority|
| - Low priority   |
+--------+---------+
         |
+--------v---------+
| Host Queues      |
| - google.com     |
| - facebook.com   |
| - etc.           |
+------------------+
```

Politeness

```
Rules:
1. Obey robots.txt
2. Rate limit per domain (1 request per second)
3. Respect crawl-delay directive
4. Avoid peak hours for small sites

Implementation:
- Per-host queue with delay
- Distributed rate limiter
- Cache robots.txt
```

Duplicate Detection

```
URL Deduplication:
- Normalize URLs (lowercase, remove fragments)
- Hash and check against seen set
- Bloom filter for memory efficiency

Content Deduplication:
- SimHash for near-duplicate detection
- MinHash for Jaccard similarity
- Store content fingerprints
```

Content Storage

```
Storage Layers:
1. Raw HTML: Object storage (S3)
2. Parsed content: Elasticsearch
3. Metadata: Database
4. URL index: Distributed hash table

Compression:
- Compress HTML before storage
- Deduplicate common templates
```

---

10. Design Dropbox/Google Drive

Requirements

Functional

- Upload/download files
- Sync files across devices
- Share files/folders
- Version history
- Offline access

Non-Functional

- Reliability (no data loss)
- Low latency sync
- Bandwidth efficiency
- Handle large files

System Design

High-Level Architecture

```
+--------+     +---------------+     +------------------+
| Client | <-> | Sync Service  | <-> | Metadata Service |
| (App)  |     +---------------+     +--------+---------+
+---+----+                                     |
    |                                   +-----v-----+
    |                                   | Metadata  |
    |                                   | Database  |
    |                                   +-----------+
    |
    |          +---------------+
    +--------> | Block Service | -----> +-------------+
               +---------------+        | Block       |
                                        | Storage(S3) |
                                        +-------------+
```

File Chunking

```
Benefits:
- Resume interrupted uploads
- Delta sync (only changed chunks)
- Deduplication across users
- Parallel upload/download

Chunk Size: 4 MB

Process:
1. Split file into chunks
2. Calculate hash of each chunk
3. Check which chunks already exist
4. Upload only new chunks
5. Store chunk list as file metadata
```

Sync Algorithm

```
Client maintains:
- Local file tree with hashes
```

```
- Last sync timestamp

Sync Process:
1. Calculate local changes since last sync
2. Pull remote changes
3. Detect conflicts
4. Apply non-conflicting changes
5. Resolve conflicts (keep both/ask user)
6. Push local changes
7. Update sync timestamp
```

Conflict Resolution

```
Scenarios:
1. Same file edited on two devices while offline
2. File deleted on one, edited on other

Resolution:
- Last writer wins (with version backup)
- Keep both versions (rename one)
- User prompt for resolution
```

Metadata Database

```sql
-- Files table
CREATE TABLE files (
    file_id UUID PRIMARY KEY,
    user_id BIGINT,
    folder_id UUID,
    name VARCHAR(255),
    size BIGINT,
    hash VARCHAR(64),
    version INT,
    is_deleted BOOLEAN,
    created_at TIMESTAMP,
    modified_at TIMESTAMP
);

-- Chunks table
CREATE TABLE chunks (
    chunk_id UUID PRIMARY KEY,
    hash VARCHAR(64) UNIQUE,
    size INT,
    storage_path TEXT
);

-- File-Chunk mapping
CREATE TABLE file_chunks (
    file_id UUID,
    chunk_id UUID,
    chunk_order INT,
    PRIMARY KEY (file_id, chunk_order)
);
```

---

11. Design a Rate Limiter

Requirements

Functional

- Limit requests per user/IP/API key
- Multiple rate limit rules
- Distributed rate limiting
- Return appropriate headers

Non-Functional

- Low latency (<1ms overhead)
- High availability
- Accurate limiting
- Fault tolerant

Algorithms

Token Bucket

```
Parameters:
- Bucket size (capacity)
- Refill rate (tokens per second)

Algorithm:
1. Each request consumes 1 token
2. If tokens available, allow request
3. If no tokens, reject request
4. Refill tokens at fixed rate

Implementation (Redis):
local tokens = redis.call('GET', key)
local last_refill = redis.call('GET', key .. ':time')
local now = ARGV[1]
local capacity = ARGV[2]
local rate = ARGV[3]

-- Calculate new tokens
local elapsed = now - last_refill
local new_tokens = math.min(capacity, tokens + elapsed * rate)

if new_tokens >= 1 then
    redis.call('SET', key, new_tokens - 1)
    redis.call('SET', key .. ':time', now)
    return 1  -- Allowed
else
    return 0  -- Rejected
end
```

Sliding Window Log

```
Store timestamp of each request:
1. Remove timestamps outside window
2. Count remaining timestamps
3. If count < limit, allow and add timestamp
4. If count >= limit, reject
```

```
Redis:
ZADD requests:{user_id} {timestamp} {request_id}
ZREMRANGEBYSCORE requests:{user_id} 0 {window_start}
ZCARD requests:{user_id}
```

Sliding Window Counter

```
Hybrid approach:
1. Divide window into smaller buckets
2. Weight current and previous bucket
3. Estimate request count

Example (1-hour window, 1-minute buckets):
- Previous minute count: 50
- Current minute count: 30
- 30 seconds into current minute
- Estimated: 50 * 0.5 + 30 = 55
```

System Design

```
                    +------------------+
                    | Rate Limit       |
                    | Configuration    |
                    +--------+---------+
                             |
+--------+     +-------+     v
| Client | --> |  LB   | --> +------------------+
+--------+     +-------+     | Rate Limiter     |
                             | Service          |
                             +--------+---------+
                                      |
                             +--------v---------+
                             | Redis Cluster    |
                             | (Counters)       |
                             +------------------+
```

Response Headers

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 47
X-RateLimit-Reset: 1640000000
Retry-After: 60

HTTP 429 Too Many Requests
```

---

12. Design a Key-Value Store

Requirements

Functional

- put(key, value)
- get(key)
- delete(key)
- Support large values

Non-Functional

- High availability
- Scalability
- Tunable consistency
- Low latency

System Design (Dynamo-style)

Architecture

```
+--------+     +------------------+     +--------+
| Client | <-> | Coordinator Node | <-> | Node 1 |
+--------+     +------------------+     +--------+
                         |               +--------+
                         +-------------> | Node 2 |
                         |               +--------+
                         +-------------> | Node 3 |
                                         +--------+
```

Data Partitioning (Consistent Hashing)

```
Ring with virtual nodes:
- Each physical node has multiple positions
- Key hashed to ring position
- Stored on next N nodes clockwise
```

Replication

```
Replication Factor: 3
Write to coordinator + 2 replicas

Quorum:
- W = 2 (write to 2 nodes)
- R = 2 (read from 2 nodes)
- W + R > N ensures consistency
```

Write Path

```
1. Client sends put(key, value)
2. Coordinator determines nodes (consistent hash)
3. Write to local commit log (WAL)
4. Write to in-memory table (memtable)
5. Replicate to other nodes
6. When memtable full, flush to SSTable
7. Return success after W acknowledgments
```

Read Path

```
1. Client sends get(key)
2. Coordinator determines nodes
3. Read from R nodes in parallel
4. Check memtable, then SSTables
5. Return most recent value (by timestamp)
6. If inconsistency, trigger read repair
```

Conflict Resolution

```
Vector Clocks:
```

```
- Track causal history
- Detect concurrent writes
- Client resolves conflicts

Last-Write-Wins:
- Simple timestamp comparison
- May lose writes
```

---

13. Design a Distributed Cache

Requirements

Functional

- get(key)
- set(key, value, ttl)
- delete(key)
- Distributed across nodes

Non-Functional

- Low latency (<1ms)
- High throughput
- High availability
- Scalability

System Design

Architecture (Redis Cluster style)

```
+--------+     +------------------+
| Client | --> | Cache Proxy      |
+--------+     | (Optional)       |
               +--------+---------+
                        |
        +---------------+---------------+
        |               |               |
   +----v----+     +----v----+     +----v----+
   | Shard 1 |     | Shard 2 |     | Shard 3 |
   | Master  |     | Master  |     | Master  |
   +----+----+     +----+----+     +----+----+
        |               |               |
   +----v----+     +----v----+     +----v----+
   | Replica |     | Replica |     | Replica |
   +---------+     +---------+     +---------+
```

Data Distribution

```
Hash Slots (Redis Cluster):
- 16,384 hash slots
- Distributed across nodes
- Key -> CRC16(key) % 16384 -> slot -> node

Client-side routing:
- Client knows slot mapping
- Direct connection to correct node
```

Cache Eviction

```
Policies:
1. LRU (Least Recently Used)
2. LFU (Least Frequently Used)
3. TTL-based expiration
4. Random eviction

Implementation:
- Doubly linked list + hash map for LRU
- Approximate LRU for efficiency
```

High Availability

```
Replication:
- Each master has 1+ replicas
- Async replication
- Automatic failover

Failure Detection:
- Gossip protocol
- Node marks as failing after timeout
- Replica promoted to master
```

---

14. Design a Search Autocomplete System

Requirements

Functional

- Return top suggestions as user types
- Rank by popularity/relevance
- Personalization (optional)

Non-Functional

- Low latency (<100ms)
- High availability
- Fresh data

System Design

Architecture

```
+--------+     +-------+     +------------------+
| Client | --> |  LB   | --> | Autocomplete     |
+--------+     +-------+     | Service          |
                             +--------+---------+
                                      |
                    +-----------------+-----------------+
                    |                                   |
             +------v------+                    +-------v-------+
             | Trie Cache  |                    | Data Pipeline |
             | (Redis)     |                    | (Update Trie) |
             +------+------+                    +-------+-------+
                    |                                   |
             +------v------+                    +-------v-------+
```

```
            | Trie Storage|                    | Query Logs    |
            | (DB/File)   |                    | (Kafka)       |
            +-------------+                    +---------------+
```

Trie Data Structure

```
Each node stores:
- Character
- Is end of word
- Top K suggestions for this prefix
- Frequency/score

Example:
        root
       / | \
      a  b  c
     /|   \
    p n    a
   /  |     \
  p   d      t
  |
  l
  |
  e (apple: 100, application: 80)
```

Building the Trie

```
Offline Process:
1. Collect search queries (logs)
2. Aggregate by query, count frequency
3. Build trie with frequencies
4. For each prefix, store top K completions
5. Serialize and distribute to servers
```

Query Flow

```
1. Client sends prefix "app"
2. Traverse trie to node for "app"
3. Return pre-computed top suggestions
4. Filter based on user context (optional)
```

Updating Suggestions

```
Approach:
1. Collect new queries in Kafka
2. Batch process periodically (hourly/daily)
3. Rebuild trie with updated frequencies
4. Rolling update to servers

Alternative (real-time):
- Sampling to update frequencies
- Bloom filter for trending detection
- Incremental trie updates
```

---

15. Design Yelp/Nearby Places

Requirements

Functional

- Search businesses by location
- Filter by category, rating, price
- View business details
- Reviews and ratings

Non-Functional

- Fast location queries (<100ms)
- Handle millions of businesses
- Support global locations

System Design

Architecture

```
+--------+     +-------+     +------------------+
| Client | --> |  LB   | --> | API Gateway      |
+--------+     +-------+     +--------+---------+
                                      |
              +-----------------------+-----------------------+
              |                       |                       |
       +------v------+         +------v------+         +------v------+
       | Search      |         | Business    |         | Review      |
       | Service     |         | Service     |         | Service     |
       +------+------+         +------+------+         +------+------+
              |                       |                       |
       +------v------+         +------v------+         +------v------+
       | Elasticsearch|         | PostgreSQL |         | MongoDB     |
       | (Geo Index) |         | + PostGIS  |         | (Reviews)   |
       +-------------+         +-------------+         +-------------+
```

Geospatial Indexing

```
Options:

1. Geohash
   - Encode lat/lng to string
   - Prefix matching for nearby
   - Easy to use with databases

2. QuadTree
   - Divide space into quadrants
   - Recursively subdivide
   - Efficient for dense areas

3. R-Tree
   - Index spatial objects
   - Used by PostGIS
   - Supports complex shapes

4. S2 Geometry (Google)
   - Sphere to cube projection
```

```
   - Hierarchical cells
   - Used by Google Maps
```

Search Query

```
Input:
- Location (lat, lng)
- Radius (km)
- Filters (category, rating, price)

Process:
1. Query geo-index for nearby businesses
2. Apply filters
3. Rank by relevance/rating/distance
4. Return paginated results

Elasticsearch Query:
{
  "query": {
    "bool": {
      "must": {"match": {"category": "restaurant"}},
      "filter": {
        "geo_distance": {
          "distance": "5km",
          "location": {"lat": 40.7128, "lon": -74.0060}
        }
      }
    }
  },
  "sort": [
    {"_geo_distance": {"location": {"lat": 40.7128, "lon": -74.0060}}}
  ]
}
```

---

16. Design a Notification System

Requirements

Functional

- Send notifications (push, SMS, email)
- Support multiple channels
- User preferences
- Rate limiting
- Scheduling

Non-Functional

- High availability
- At-least-once delivery
- Scalability (millions of notifications)
- Low latency

System Design

```
+-----------+     +-----------------+     +-------------------+
```

```
| Services | --> | Notification   | --> | Message Queue    |
|          |     | API            |     | (Kafka)          |
+----------+     +----------------+     +--------+---------+
                                                 |
         +---------------------------------------+
         |               |               |       |
   +-----v-----+   +-----v-----+   +-----v-----+ +-----v-----+
   | Push      |   | Email     |   | SMS       | | In-App    |
   | Worker    |   | Worker    |   | Worker    | | Worker    |
   +-----+-----+   +-----+-----+   +-----+-----+ +-----+-----+
         |               |               |             |
   +-----v-----+   +-----v-----+   +-----v-----+ +-----v-----+
   | APNS/FCM  |   | SendGrid  |   | Twilio    | | WebSocket |
   +-----------+   +-----------+   +-----------+ +-----------+
```

Notification Flow

```
1. Service sends notification request
2. Validate and enrich (user preferences)
3. Route to appropriate channel queues
4. Workers process messages
5. Send via third-party providers
6. Track delivery status
7. Handle failures (retry, DLQ)
```

User Preferences

```
{
  "user_id": "123",
  "channels": {
    "push": {"enabled": true, "quiet_hours": "22:00-07:00"},
    "email": {"enabled": true, "digest": true},
    "sms": {"enabled": false}
  },
  "categories": {
    "marketing": {"push": false, "email": true},
    "transactional": {"push": true, "email": true}
  }
}
```

---

17. Design an Online Ticketing System

Requirements

Functional

- Browse events
- View seat availability
- Book tickets
- Payment processing
- QR code generation

Non-Functional

- Handle high concurrency (ticket rush)
- Prevent overselling

- Low latency booking
- Fair queuing

System Design

Handling Concurrency

```
Approaches:

1. Pessimistic Locking
   - Lock seats during booking
   - Hold for limited time (5 min)
   - Release if not confirmed

2. Optimistic Locking
   - Version number on seats
   - Check version on commit
   - Retry on conflict

3. Distributed Lock (Redis)
   - SETNX for seat lock
   - TTL to auto-release
   - Redlock for distributed
```

Booking Flow

```
1. User selects seats
2. Temporarily reserve seats (5 min)
   - SET seat:123 "reserved" EX 300
3. Process payment
4. If success:
   - Confirm reservation
   - Generate tickets
5. If failure:
   - Release reservation
```

Queue System for High Demand

```
For popular events:
1. User joins virtual queue
2. Assign queue position
3. Notify when turn arrives
4. Limited time to complete booking
5. Release spot if timeout
```

---

18. Design Pastebin

Requirements

Functional

- Create paste with content
- Generate unique URL
- Optional expiration
- Access control (public/private)

Non-Functional

- High availability
- Low latency access
- Handle large pastes

System Design

```
+--------+     +-------+     +------------------+
| Client | --> |  LB   | --> | Paste Service    |
+--------+     +-------+     +---------+---------+
                                       |
                     +-----------------+-----------------+
                     |                                   |
             +------v------+                    +-------v-------+
             | Metadata    |                    | Content       |
             | (MySQL)     |                    | Storage (S3)  |
             +-------------+                    +---------------+
```

Key Generation

```
Options:
1. Base62 encoding of auto-increment ID
2. Pre-generated keys (KGS)
3. Hash of content (deduplication)

Recommended: Pre-generated keys
- No collision handling
- Fast paste creation
- Predictable length
```

---

19. Design a Unique ID Generator

Requirements

Functional

- Generate unique IDs
- IDs should be sortable by time
- 64-bit numeric IDs

Non-Functional

- High availability
- Low latency (<1ms)
- No coordination needed

Approaches

1. UUID

```
UUID v4: Random 128-bit
- Simple
- No coordination
- Not sortable
- Large (128 bits)
```

2. Database Auto-Increment

```
Multiple DB with different starting points:
- DB1: 1, 3, 5, 7...
- DB2: 2, 4, 6, 8...

Pros: Simple
Cons: Single point of failure, coordination needed
```

3. Twitter Snowflake

```
64-bit ID structure:
+----------+------------+------------+--------------+
| 1 bit    | 41 bits    | 10 bits    | 12 bits      |
| (unused) | timestamp  | machine ID | sequence num |
+----------+------------+------------+--------------+

- Timestamp: milliseconds since epoch (~69 years)
- Machine ID: supports 1024 machines
- Sequence: 4096 IDs per ms per machine

Total: 4M IDs per second per machine
```

Implementation (Snowflake)

```javascript
class SnowflakeGenerator {
    constructor(machineId) {
        this.machineId = machineId;
        this.sequence = 0;
        this.lastTimestamp = -1;
        this.epoch = 1609459200000; // Custom epoch
    }

    nextId() {
        let timestamp = Date.now() - this.epoch;

        if (timestamp === this.lastTimestamp) {
            this.sequence = (this.sequence + 1) & 0xFFF;
            if (this.sequence === 0) {
                // Wait for next millisecond
                timestamp = this.waitNextMillis(timestamp);
            }
        } else {
            this.sequence = 0;
        }

        this.lastTimestamp = timestamp;

        return (BigInt(timestamp) << 22n) |
               (BigInt(this.machineId) << 12n) |
               BigInt(this.sequence);
    }

    waitNextMillis(timestamp) {
        while (timestamp <= this.lastTimestamp) {
            timestamp = Date.now() - this.epoch;
        }
```

```
        return timestamp;
    }
}
```

---

20. Design a Payment System

Requirements

Functional

- Process payments
- Multiple payment methods
- Refunds
- Transaction history
- Fraud detection

Non-Functional

- High availability (99.999%)
- Strong consistency
- PCI compliance
- Idempotency

System Design

```
+--------+     +---------------+     +------------------+
| Client | --> | API Gateway   | --> | Payment Service  |
+--------+     | (Rate Limit,  |     |                  |
               | Auth)         |     +--------+---------+
               +---------------+              |
                                     +---------+---------+
                                     |                   |
                              +------v------+     +------v------+
                              | PSP Router  |     | Ledger      |
                              | (Stripe,etc)|     | Service     |
                              +------+------+     +------+------+
                                     |                   |
                              +------v------+     +------v------+
                              | Payment     |     | Transaction |
                              | Providers   |     | Database    |
                              +-------------+     +-------------+
```

Payment Flow

```
1. Client initiates payment
2. Generate idempotency key
3. Validate request
4. Create pending transaction
5. Route to payment provider
6. Process with provider
7. Update transaction status
8. Update ledger
9. Return result
10. Async: Notify, reconcile
```

Idempotency

```
Every payment request includes idempotency key:
- Client generates unique key
- Server stores key -> result mapping
- Duplicate request returns cached result

Implementation:
1. Check if key exists in cache/DB
2. If exists, return stored result
3. If not, process and store result
```

Double-Entry Ledger

```
Every transaction creates two entries:
- Debit from source account
- Credit to destination account
- Sum of all entries = 0 (balanced)

Transaction:
| ID | Account    | Type   | Amount |
|----|------------|--------|--------|
| 1  | Customer A | Debit  | -100   |
| 1  | Merchant B | Credit | +100   |
```

Reconciliation

```
Daily process:
1. Fetch transactions from our system
2. Fetch settlement reports from providers
3. Match transactions
4. Flag discrepancies
5. Investigate and resolve
```

Quick Reference: System Design Checklist

For Every Design

1. **Clarify Requirements**
   - Functional requirements
   - Non-functional requirements
   - Scale and constraints
2. **Capacity Estimation**
   - Users (DAU/MAU)
   - Traffic (QPS)
   - Storage
   - Bandwidth
3. **High-Level Design**
   - Core components
   - Data flow
   - API design
4. **Detailed Design**
   - Database schema
   - Algorithms
   - Data structures
5. **Scale and Optimize**

- Bottlenecks
- Caching
- Sharding
- CDN

6. **Reliability**
   - Replication
   - Failover
   - Monitoring

End of System Design Interview Questions Guide